

Incremental Residue-Class Method for Generating Prime Numbers

Ralf Sieg

2026

Abstract

This paper describes an incremental method for generating prime numbers that operates without divisions, multiplications, or explicit modulo computations. Instead, residue values modulo already known primes are incrementally advanced. The method is simple, state-lean, and infinitely extensible, requiring only the minimal information necessary for incremental progression. Although asymptotically slower than classical sieving techniques, the method offers conceptual simplicity and practical advantages in streaming and embedded environments where incremental, low-state computation is essential. A complete complexity analysis is provided.

1 Introduction

The method presented here generates prime numbers by sequentially iterating through all natural numbers starting from 2, without performing divisions, multiplications, or modulo operations explicitly. Instead, the residue values of division by already known primes are incrementally advanced. This yields a simple, infinitely extensible procedure for prime detection.

Digits are written in quotation marks (“0”, “1”, ...), while numbers are written without quotation marks (2, 3, 4 ...).

2 Basic Idea: A Residue-Class Vector over Prime Numbers

The method does not use a positional numeral system such as the decimal system, but rather a residue-class vector, referred to as the *prime-number system*. Each position corresponds to a prime number p and stores the residue of the current number $n \bmod p$.

For each prime p , there exists a position with a digit range from “0” to “ $p - 1$ ”.

Thus, each position represents the value $n \bmod p$.

3 Incrementing in the Prime-Number System

When transitioning from n to $n + 1$, the following holds:

$$(n + 1) \bmod p = (n \bmod p + 1) \bmod p.$$

This yields the central rule:

All positions of the system are incremented by 1. When a position reaches its maximum value, it wraps around to “0”. New positions always begin at “0”.

4 Prime Detection

A number n is prime if and only if it is not divisible by any smaller prime. In the prime-number system, this means:

If, after incrementing, no position has the value “0”, then the number is prime.

For each newly discovered prime, a new position with digit “0” is added. Thus, upon reaching a number n , all primes $\leq n$ are already present.

5 Representation in the Decimal System

To visualize the process, the decimal representation and the residue-class vector are shown in parallel.

Decimal	Prime associated with the position				
	2	3	5	7	11
2	0				
3	1	0			
4	0	1			
5	1	2	0		
6	0	0	1		
7	1	1	2	0	
8	0	2	3	1	
9	1	0	4	2	
10	0	1	0	3	
11	1	2	1	4	0

6 Complexity Analysis

For each number n , the method advances all residue positions corresponding to the primes known so far.

6.1 Time Complexity

The number of positions equals the number of primes $\leq n$:

$$\pi(n) \sim \frac{n}{\ln n}.$$

Total cost up to N :

$$T(N) = \sum_{n=2}^N \pi(n) = O(N \cdot \pi(N)) = O\left(\frac{N^2}{\ln N}\right).$$

Interpretation

- Quadratic divided by a logarithm.
- Slower than the sieve of Eratosthenes ($O(N \log \log N)$).
- Extremely simple, incremental, state-lean, and infinitely extensible.

6.2 Space Complexity

One position per prime:

$$S(N) = \pi(N) = O\left(\frac{N}{\ln N}\right).$$

6.3 Comparison with Classical Methods

Method	Time	Space	Remarks
Incremental residue-class method	$O(N^2 / \ln N)$	$O(N / \ln N)$	Simple, incremental, no division
Sieve of Eratosthenes	$O(N \log \log N)$	$O(N)$	Very fast, not incremental
Trial division	$O(N\sqrt{N})$	$O(1)$	Extremely slow
Segmented sieve	$O(N \log \log N)$	$O(N)$	Memory-optimized

7 Strengths and Weaknesses

7.1 Strengths

- No division, multiplication, or modulo.
- Fully incremental.
- Indefinitely extensible.
- Very simple implementation.

7.2 Weaknesses

- Asymptotically slower than modern sieves.
- Not competitive for very large ranges.

8 Conclusion

The incremental residue-class method is an unusually simple, fully incremental prime-generation procedure. It is not optimized for maximum speed, but offers unique advantages for streaming and embedded scenarios.

License

Licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). Full license text: <https://creativecommons.org/licenses/by/4.0/legalcode>